Formal Algorithms for GPT-2 Small Model

Predicting t_9 from $t_1, \ldots, t_8$

One Forward Pass of DTransformer — Algorithm 10

Devi Prasad M

September, 2026

Setup

The prompt is $x = (t_1, \ldots, t_8)$, so $\ell = \text{length}(x) = 8$. We follow one call of Algorithm 10 and read the causal mask one query position at a time.

ℓ	8	d_e, d_{in}, d_{out}	768
$\ell_{\max}$	1024	H	12
L	12	$d_{\text{attn}}, d_{\text{mid}}$	64
N_V	50257	d_{mlp}	3072

Notation. The paper prints line 6 of Algorithm 10 as MHAttention($\tilde{\mathbf{X}} \mid \mathcal{W}_l$, Mask). Algorithm 5 takes two sequences, so we write the call in full as MHAttention($\tilde{\mathbf{X}}, \tilde{\mathbf{X}} \mid \mathcal{W}_l$, Mask): in self-attention the context $\tilde{\mathbf{Z}}$ is $\tilde{\mathbf{X}}$ itself.

Mask convention. As in the paper, Mask $\in \{0,1\}^{\ell_z \times \ell_x}$: rows are key positions t_z, columns are query positions t_x, and softmax runs down each column of $\mathbf{S}$.

query (this column) visible key/value masked (future) token this column predicts

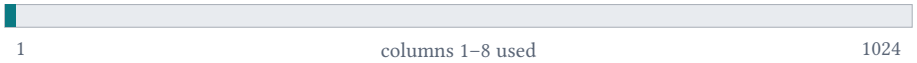
Line 2, embeddings: build X from 8 tokens and 8 positions

2 for $t \in [\ell]$: $\mathbf{e}_t \leftarrow \mathbf{W_e}[:, x[t]] + \mathbf{W_p}[:, t]$



$\ell \leftarrow \text{length}(x)$	8
Token columns read	$\mathbf{W_e}[:, x[1]], \ldots, \mathbf{W_e}[:, x[8]]$
Position columns read	$\mathbf{W_p}[:, 1{:}8]$ (8 of 1024)
$\mathbf{e}_t$	768×1 each
$\mathbf{X} = [\mathbf{e}_1, \ldots, \mathbf{e}_8]$	768×8
Padding to $\ell_{\max}$	none

Position embedding columns read from $\mathbf{W_p} \in \mathbb{R}^{768\times1024}$:



All eight tokens go in together. Each gets its token embedding plus the position embedding for its slot. Only columns 1–8 of $\mathbf{W_p}$ are read. $\ell_{\max} = 1024$ is the number of columns $\mathbf{W_p}$ has, the longest input the model accepts. Nothing is padded to 1024.

From here on every layer works on $\mathbf{X} \in \mathbb{R}^{768\times8}$, so every attention call has $\ell_x = \ell_z = 8$.

$$6 \quad \mathbf{X} \leftarrow \mathbf{X} + \mathrm{MHAttention}(\tilde{\mathbf{X}}, \tilde{\mathbf{X}} \mid \mathcal{W}_l,\ \mathrm{Mask}[t_z, t_x] = [[t_z \le t_x]])$$

Runs in each of the $L = 12$ layers.

Line 6, query column $t_x = 1$ of 8: which tokens can t_1 see?



$z\backslash x$	**1**	2	3	4	5	6	7	8
1	**1**	1	1	1	1	1	1	1
2	**0**	1	1	1	1	1	1	1
3	**0**	0	1	1	1	1	1	1
4	**0**	0	0	1	1	1	1	1
5	**0**	0	0	0	1	1	1	1
6	**0**	0	0	0	0	1	1	1
7	**0**	0	0	0	0	0	1	1
8	**0**	0	0	0	0	0	0	1

Column $t_x = 1$: row 1 is 1; rows 2–8 get $S = -\infty$ before the softmax.

ℓ_x (queries)	8
ℓ_z (keys, values)	8
$\tilde{\mathbf{X}} = \tilde{\mathbf{Z}}$	768×8
Per head: $\mathbf{Q}, \mathbf{K}, \mathbf{V}$	64×8 each
$\mathbf{S} = \mathbf{K}^\top \mathbf{Q}$	8×8 $(\ell_z \times \ell_x)$
Query column viewed	$t_x = 1$
Keys it can see	$t_z = 1$ (itself only)
Masked keys	$t_z = 2, \dots, 8$
$\mathbf{P}[:, 1]$ estimates	$\hat{P}(x[2] \mid x[1{:}1])$
Is $x[2]$ already known?	yes: t_2

All eight query columns are computed at once, in one matrix product per head. Going through them one at a time is only a way to read the mask.

Query $t_x = 1$ is t_1. The mask keeps keys with $t_z \le 1$, so it can attend only to itself and never sees $t_2, \dots, t_8$.

Because this holds in every layer, column 1 of the output depends only on $x[1{:}1]$. So $\mathbf{P}[:, 1]$ is a prediction of $x[2]$.

But $x[2] = t_2$ is already in the prompt. At inference this column is computed and thrown away. In training it is one of the 8 loss terms.

Line 6, query column $t_x = 2$ of 8: which tokens can t_2 see?



$z\backslash x$	1	**2**	3	4	5	6	7	8
1	1	**1**	1	1	1	1	1	1
2	0	**1**	1	1	1	1	1	1
3	0	**0**	1	1	1	1	1	1
4	0	**0**	0	1	1	1	1	1
5	0	**0**	0	0	1	1	1	1
6	0	**0**	0	0	0	1	1	1
7	0	**0**	0	0	0	0	1	1
8	0	**0**	0	0	0	0	0	1

Column $t_x = 2$: rows 1–2 are 1; rows 3–8 get $S = -\infty$ before the softmax.

ℓ_x (queries)	8
ℓ_z (keys, values)	8
$\tilde{\mathbf{X}} = \tilde{\mathbf{Z}}$	768×8
Per head: $\mathbf{Q}, \mathbf{K}, \mathbf{V}$	64×8 each
$\mathbf{S} = \mathbf{K}^\top \mathbf{Q}$	8×8 $(\ell_z \times \ell_x)$
Query column viewed	$t_x = 2$
Keys it can see	$t_z = 1, \dots, 2$ (2 of 8)
Masked keys	$t_z = 3, \dots, 8$
$\mathbf{P}[:, 2]$ estimates	$\hat{P}(x[3] \mid x[1{:}2])$
Is $x[3]$ already known?	yes: t_3

Query $t_x = 2$ is t_2. The mask keeps keys with $t_z \le 2$, so it attends to $t_1, \dots, t_2$ and never sees $t_3, \dots, t_8$.

Because this holds in every layer, column 2 of the output depends only on $x[1{:}2]$. So $\mathbf{P}[:, 2]$ is a prediction of $x[3]$.

But $x[3] = t_3$ is already in the prompt. At inference this column is computed and thrown away. In training it is one of the 8 loss terms.

Line 6, query column $t_x = 3$ of 8: which tokens can t_3 see?



$z\backslash x$	1	2	**3**	4	5	6	7	8
1	1	1	**1**	1	1	1	1	1
2	0	1	**1**	1	1	1	1	1
3	0	0	**1**	1	1	1	1	1
4	0	0	**0**	1	1	1	1	1
5	0	0	**0**	0	1	1	1	1
6	0	0	**0**	0	0	1	1	1
7	0	0	**0**	0	0	0	1	1
8	0	0	**0**	0	0	0	0	1

Column $t_x = 3$: rows 1–3 are 1; rows 4–8 get $S = -\infty$ before the softmax.

ℓ_x (queries)	8
ℓ_z (keys, values)	8
$\tilde{\mathbf{X}} = \tilde{\mathbf{Z}}$	768×8
Per head: $\mathbf{Q}, \mathbf{K}, \mathbf{V}$	64×8 each
$\mathbf{S} = \mathbf{K}^\top \mathbf{Q}$	8×8 $(\ell_z \times \ell_x)$
Query column viewed	$t_x = 3$
Keys it can see	$t_z = 1, \dots, 3$ (3 of 8)
Masked keys	$t_z = 4, \dots, 8$
$\mathbf{P}[:, 3]$ estimates	$\hat{P}(x[4] \mid x[1{:}3])$
Is $x[4]$ already known?	yes: t_4

Query $t_x = 3$ is t_3. The mask keeps keys with $t_z \le 3$, so it attends to $t_1, \dots, t_3$ and never sees $t_4, \dots, t_8$.

Because this holds in every layer, column 3 of the output depends only on $x[1{:}3]$. So $\mathbf{P}[:, 3]$ is a prediction of $x[4]$.

But $x[4] = t_4$ is already in the prompt. At inference this column is computed and thrown away. In training it is one of the 8 loss terms.

⋮

Line 6, query column $t_x = 7$ of 8: which tokens can t_7 see?



$z\backslash x$	1	2	3	4	5	6	**7**	8
1	1	1	1	1	1	1	**1**	1
2	0	1	1	1	1	1	**1**	1
3	0	0	1	1	1	1	**1**	1
4	0	0	0	1	1	1	**1**	1
5	0	0	0	0	1	1	**1**	1
6	0	0	0	0	0	1	**1**	1
7	0	0	0	0	0	0	**1**	1
8	0	0	0	0	0	0	**0**	1

Column $t_x = 7$: rows 1–7 are 1; row 8 gets $S = -\infty$ before the softmax.

ℓ_x (queries)	8
ℓ_z (keys, values)	8
$\tilde{\mathbf{X}} = \tilde{\mathbf{Z}}$	768×8
Per head: $\mathbf{Q}, \mathbf{K}, \mathbf{V}$	64×8 each
$\mathbf{S} = \mathbf{K}^\top \mathbf{Q}$	8×8 $(\ell_z \times \ell_x)$
Query column viewed	$t_x = 7$
Keys it can see	$t_z = 1, \dots, 7$ (7 of 8)
Masked keys	$t_z = 8$
$\mathbf{P}[:, 7]$ estimates	$\hat{P}(x[8] \mid x[1{:}7])$
Is $x[8]$ already known?	yes: t_8

Query $t_x = 7$ is t_7. The mask keeps keys with $t_z \le 7$, so it attends to $t_1, \dots, t_7$ and never sees t_8.

Because this holds in every layer, column 7 of the output depends only on $x[1{:}7]$. So $\mathbf{P}[:, 7]$ is a prediction of $x[8]$.

But $x[8] = t_8$ is already in the prompt. At inference this column is computed and thrown away. In training it is one of the 8 loss terms.

Line 6, query column $t_x = 8$ of 8: which tokens can t_8 see?



$z\backslash x$	1	2	3	4	5	6	7	**8**
1	1	1	1	1	1	1	1	**1**
2	0	1	1	1	1	1	1	**1**
3	0	0	1	1	1	1	1	**1**
4	0	0	0	1	1	1	1	**1**
5	0	0	0	0	1	1	1	**1**
6	0	0	0	0	0	1	1	**1**
7	0	0	0	0	0	0	1	**1**
8	0	0	0	0	0	0	0	**1**

Column $t_x = 8$: rows 1–8 are 1; no row is masked.

ℓ_x (queries)	8
ℓ_z (keys, values)	8
$\tilde{\mathbf{X}} = \tilde{\mathbf{Z}}$	768×8
Per head: $\mathbf{Q}, \mathbf{K}, \mathbf{V}$	64×8 each
$\mathbf{S} = \mathbf{K}^\top \mathbf{Q}$	8×8 $(\ell_z \times \ell_x)$
Query column viewed	$t_x = 8$
Keys it can see	$t_z = 1, \dots, 8$ (8 of 8)
Masked keys	none
$\mathbf{P}[:, 8]$ estimates	$\hat{P}(x[9] \mid x[1{:}8])$
Is $x[9]$ already known?	no: this is t_9

Query $t_x = 8$ is t_8. The mask keeps keys with $t_z \le 8$, so it attends to $t_1, \dots, t_8$, the whole prompt.

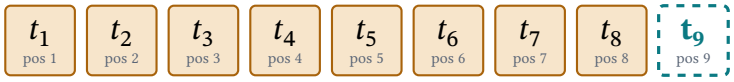
Because this holds in every layer, column 8 of the output depends only on $x[1:8]$. So $\mathbf{P}[:, 8]$ is a prediction of $x[9]$.

$x[9]$ is not in the prompt. This is the only column inference needs: $\mathbf{P}[:, 8] = \hat{P}(x[9] \mid t_1, \ldots, t_8)$.

Line 11, output: sample t_9 from column 8 of P

11	return $\mathbf{P} = \text{softmax}(\mathbf{W_u X})$

Then Algorithm 14, line 4 with $i = 1$: $\mathbf{p} \leftarrow \mathbf{P}[:, \ell + i - 1] = \mathbf{P}[:, 8]$.



$\mathbf{X}$ after final layer norm	768×8
$\mathbf{W_u}$	50257×768
$\mathbf{P} = \text{softmax}(\mathbf{W_u X})$	50257×8
Columns 1–7	computed, not used at inference
Column used	$\mathbf{P}[:, 8] = \hat{P}(x[9] \mid x[1:8])$
Sample	$t_9 \sim \mathbf{q} \propto \mathbf{P}[:, 8]^{1/\tau}$

$\mathbf{P}$ has one column per input position. Column t is the model's distribution over the next token after $x[1:t]$.

At inference only the last column matters. Columns 1–7 predict tokens we already have, $t_2, \ldots, t_8$.

In training, the same pass is used differently: all 8 columns are scored against their known next tokens, giving 8 loss terms from one forward pass. The causal mask is what makes that valid.

Appendix: the same mask in the GPT-2 code

The openai/gpt-2 code (`src/model.py`) stores the mask transposed: rows are queries i (destination), columns are keys j (source), both 0-indexed. `attention_mask(nd, ns)` sets

$$m[i, j] = [[\, i \geq j - n_s + n_d \,]],$$

which is $i \geq j$ when $n_d = n_s = 8$. The softmax runs along each row. Below, token t_3 (row $i = 2$) is highlighted; compare with the paper-layout matrix at $t_x = 3$.

$i \backslash j$	0	1	2	3	4	5	6	7
0	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0
2	**1**	**1**	**1**	**0**	**0**	**0**	**0**	**0**
3	1	1	1	1	0	0	0	0
4	1	1	1	1	1	0	0	0
5	1	1	1	1	1	1	0	0
6	1	1	1	1	1	1	1	0
7	1	1	1	1	1	1	1	1

An eight-token prompt.

Let

$$\mathbf{x} = (t_1, t_2, \dots, t_8), \qquad \ell_x = \text{length}(\mathbf{x}) = 8.$$

A call to $P = \text{DTransformer}(\mathbf{x} \mid \hat{\theta})$ produces eight output columns. Column $P[:, j]$ gives a distribution for the token *after* the prefix $(t_1, \dots, t_j)$.

Position j	Initial column $X^{(0)}[:, j]$	Visible positions	Next-token distribution
1	$W_e[:, t_1] + W_p[:, 1]$	$\{1\}$	$P[:, 1]$
2	$W_e[:, t_2] + W_p[:, 2]$	$\{1, 2\}$	$P[:, 2]$
3	$W_e[:, t_3] + W_p[:, 3]$	$\{1, 2, 3\}$	$P[:, 3]$
		$\vdots$	
7	$W_e[:, t_7] + W_p[:, 7]$	$\{1, \dots, 7\}$	$P[:, 7]$
8	$W_e[:, t_8] + W_p[:, 8]$	$\{1, \dots, 8\}$	$P[:, 8]$

The causal attention mask.

Rows index a potential source (context) position r; columns index the position j whose representation is being updated:

$$\text{Mask}[r, j] = \begin{cases} 1, & r \le j, \\ 0, & r > j. \end{cases}$$

For this prompt, the complete mask is

$$\begin{array}{c} \\ r{=}1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \end{array} \begin{array}{c} \begin{array}{cccccccc} j{=}1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \end{array} \\ \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \end{array}$$

Thus column $j = 3$ can attend only to positions $1, 2, 3$, whereas column $j = 8$ can attend to all eight positions. A zero forbids an attention link; it does not replace an input token.

Predicting t_9. After all transformer layers and the final unembedding, Algorithm 14 takes

$$\mathbf{p} = P[:, \ell_x] = P[:, 8] = \widehat{\Pr}_{\hat{\theta}}(\,\cdot \mid t_1, \dots, t_8).$$

The next token t_9 is selected or sampled from this distribution. There is no t_9 input column in this forward pass.